SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192324035
Name	Shri Shanmuga Priya K

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**

Question Count: 5

Code of Conduct

I Shri Shanmuga Priya K (192324035) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

Question 1: Uneven Input Splits and Idle Reducers in MapReduce

1. Problem Statement & System Context

In distributed computing frameworks like Apache Hadoop MapReduce, linear scalability depends heavily on uniform data partitioning. When input splits vary dramatically in size or when intermediate keys are unevenly distributed, the system experiences computational stragglers and data skew. While a few tasks perform massive computations, other allocated worker nodes sit idle, consuming cluster resources without contributing to throughput and ultimately causing job failures and SLA violations.

2. Root Cause & Architectural Error Analysis

- Non-Splittable Compression Formats: Input files compressed using non-splittable formats (such as standard GZIP without an index, ZIP, or raw TAR archives) cannot be broken down across logical HDFS block boundaries. As a consequence, Hadoop is forced to assign an entire multi-gigabyte file to a single Mapper, bypassing data parallelism completely.
- Severe Intermediate Key Skew (Partitioning Anomaly): Hadoop's default partitioner determines Reducer assignment by computing a hash of the key modulo the number of Reducers. If the data contains dominant keys (such as null values, blank fields, or default category IDs), all corresponding key-value pairs are routed to a single Reducer. That single Reducer becomes overloaded, while all other Reducers finish almost instantly and remain idle.
- The Small Files Problem: If input datasets consist of millions of tiny files significantly smaller than the default HDFS block size, the framework generates an individual Mapper for each file. This creates massive metadata and container management overhead, resulting in uneven split assignment and downstream Reducer starvation.
- Improper Input Split Configuration: Misconfigured maximum and minimum split size parameters can lead to logical split calculations that fail to align with the underlying storage blocks, creating jagged partitions across worker nodes.

3. Impact on System Performance

- Resource Inefficiency & Waste: Reducer containers allocated by the cluster manager hold dedicated CPU and RAM allocations while waiting for the single overloaded Mapper or processing nothing at all.
- Task Timeouts & Job Failures: Overloaded tasks frequently exceed the task heartbeat threshold, triggering automated task aborts and costly retries that eventually fail the entire job.
- Severe Pipeline Latency: Total job execution time is bound to the single slowest task in the cluster, completely undermining horizontal scaling.

4. Corrective Actions & Architectural Remediation

- Transition to Splittable File Formats: Store data using natively splittable, binary, or columnar formats such as Apache Parquet, Apache ORC, or Apache Avro. If flat text files must be preserved, compress them with block-splittable codecs like Bzip2.

- Logical Split Merging: Configure the job to use a combined file input format that logically groups multiple small files or sub-block fragments into unified, evenly sized splits matching the target HDFS block size before launching Mappers.
- Key Salting Technique: Mitigate data skew at the partitioner by appending a deterministic or pseudo-random prefix/suffix to high-frequency intermediate keys during the Map phase. This scatters dominant keys uniformly across all available Reducers, followed by a secondary consolidation step.
- Custom Partitioner Implementation: Replace the default hash partitioner with an intelligent, content-aware partitioner that identifies heavily skewed keys and distributes them across a dedicated subset of Reducers while balancing standard traffic across the remaining workers.
- Map-Side Aggregation (Combiners): Enforce local mini-reducers (Combiners) directly on the Mapper nodes. Combiners aggregate intermediate key-value pairs in memory before network transmission, dramatically cutting the volume of skewed data moving into the shuffle-and-sort phase.

Question 2: HDFS Data Unavailability After a Single Node Failure

1. Problem Statement & System Context

The core design philosophy of Hadoop Distributed File System (HDFS) is built on the assumption that commodity hardware components fail continuously. A resilient distributed storage layer must sustain single or multiple node crashes without interrupting file read/write operations. When a single DataNode or master server outage causes immediate data unavailability, it signals a critical misconfiguration in replication policies, topology mapping, or high-availability control planes.

2. Root Cause & Architectural Error Analysis

- Under-Replication Policy: The cluster-wide default replication factor or specific directory replication is set to 1. In this state, HDFS maintains only one physical copy of a block. If the host DataNode crashes, loses power, or experiences drive failure, all blocks residing on that node become immediately unreachable.
- Single Point of Failure (SPOF) on the NameNode: The cluster runs a traditional standalone NameNode architecture without a hot-standby failover. If this single master server experiences hardware failure, network isolation, or prolonged Java Virtual Machine garbage collection pauses, the global namespace metadata becomes inaccessible, paralyzing all client operations across the entire cluster.
- Missing or Broken Rack Awareness Topology: When rack awareness is unconfigured, HDFS defaults to treating all DataNodes as if they reside in the same physical rack. If the replication policy inadvertently places all replicas on machines serviced by the same top-of-rack network switch or power strip, a single hardware or network failure takes down all redundant copies simultaneously.

- Corrupted or Incomplete Block Reports: A node failure coupled with delayed heartbeat detection or disk channel errors can cause the NameNode to prematurely mark surviving replicas as corrupt or missing if synchronization timeouts are poorly tuned.

3. Impact on System Performance

- Service Outage & System Downtime: Applications attempting to access affected files throw missing-block or connection exceptions, halting dependent analytics and ingestion workflows.
- Permanent Data Loss Risk: For datasets configured with a single replica, a catastrophic disk failure on that single DataNode results in unrecoverable data loss.
- Read Bottlenecks: Losing a node in an under-replicated cluster eliminates the system's ability to balance read streams across parallel storage nodes.

4. Corrective Actions & Architectural Remediation

- Enforce Fault-Tolerant Replication Defaults: Establish a strict cluster-wide baseline replication factor of 3 for all production datasets. Continuously run background file system consistency audits to detect and reconstruct under-replicated or corrupt blocks automatically.
- Deploy High Availability (HA) NameNode Architecture: Implement an Active/Standby NameNode topology backed by a Quorum Journal Manager cluster and ZooKeeper-based automated failover controllers. This ensures that if the Active NameNode becomes unresponsive, the Standby NameNode immediately assumes namespace leadership with zero downtime.
- Configure Deterministic Rack Awareness: Deploy a validated network topology script that enforces HDFS rack-placement rules: placing the first block copy on a local node, the second copy on a separate remote rack, and the third copy on a different node within that same remote rack. This protects data integrity against both node-level and rack-level outages.
- Tune Failure Detection and Heartbeat Thresholds: Calibrate DataNode heartbeat intervals, stale-state thresholds, and NameNode recovery parameters so the cluster quickly identifies dead nodes and immediately commands surviving DataNodes to replicate missing blocks to restore target replication levels.

Question 3: Duplicate Record Generation in Apache NiFi / ETL Pipelines

1. Problem Statement & System Context

Enterprise ETL pipelines built with Apache NiFi are responsible for moving data from diverse sources into analytical data stores, data lakes, and data warehouses. When duplicate records infiltrate the target analytics layer, it directly invalidates historical reporting, inflates aggregation metrics, causes erroneous machine-learning inferences, and damages downstream business intelligence integrity.

2. Root Cause & Architectural Error Analysis

- At-Least-Once Delivery Semantics with Unhandled Retries: Apache NiFi guarantees at-least-once message processing. If downstream systems experience transient network timeouts, slow disk

writes, or temporary connection resets, NiFi rolls back the transaction and retries the flow. Without upstream deduplication or downstream idempotency, records already transmitted prior to the timeout are resent and appended a second time.

- Non-Atomic Batch Processing: ETL pipelines processing records in large micro-batches often fail mid-stream due to data validation errors or resource starvation. If the ingestion mechanism retries the entire batch rather than isolating failed records, the successfully processed subset of records is ingested again.
- Non-Idempotent, Append-Only Storage Sinks: The destination analytics tables are configured for blind append operations without unique key constraints, deduplication barriers, or upsert capabilities. Every write instruction is treated as a new row insertion regardless of duplicate primary keys.
- Uncommitted Change Data Capture (CDC) Offsets: In database replication flows, if the CDC ingestion processor fails or restarts before successfully persisting its database transaction log coordinates (such as MySQL binlog coordinates or log sequence numbers) to persistent storage, it re-reads and re-ingests previously processed database events upon recovery.

3. Impact on System Performance

- Analytical Inaccuracy: Core business metrics (e.g., revenue totals, unique customer counts, daily transactions) become severely distorted due to double-counting.
- Storage and Compute Overhead: Redundant records waste physical disk space, bloat indexing overhead, and increase query runtimes across data lake tables.
- Complex Data Cleansing Costs: Rectifying corrupted datasets requires running expensive offline deduplication jobs and data reconciliation procedures.

4. Corrective Actions & Architectural Remediation

- Implement In-Flight Stream Deduplication: Embed deduplication processors within the NiFi dataflow backed by a centralized distributed cache or fast in-memory key-value store. Calculate cryptographic hashes across unique composite business keys and check incoming payloads against a sliding-window cache before forwarding data downstream.
- Adopt Transactional Table Formats (Lakehouses): Migrate analytical sinks from naive append-only storage to modern ACID transactional table formats such as Apache Iceberg, Delta Lake, or Apache Hudi. These formats natively support atomic merge-and-upsert operations, ensuring existing records are updated while new records are inserted cleanly.
- Enforce Persistent State Management for CDC: Ensure that all CDC processors use external, durable state repositories (such as ZooKeeper or durable state managers) so that database log offsets are committed atomically along with successful data delivery.
- Design End-to-End Idempotent Pipelines: Structure ingestion workflows such that re-running a pipeline across the same time window produces the exact same end state without side effects, using natural deduplication keys, target record watermarking, and staging partitions.

Question 4: Multi-User YARN Cluster Resource Starvation

1. Problem Statement & System Context

Apache Hadoop YARN coordinates compute resources—specifically CPU virtual cores and physical memory—across heterogeneous workloads including batch ETL, interactive SQL queries, and machine learning jobs. In multi-tenant environments where numerous users and teams submit tasks concurrently, inadequate resource scheduling leads to resource monopolization, unfair queue starvation, and catastrophic SLA failures for mission-critical jobs.

2. Root Cause & Architectural Error Analysis

- Default FIFO Scheduler Deployment: The cluster operates using the default First-In, First-Out scheduler. Under this policy, the first submitted job—regardless of whether it is an unoptimized ad-hoc query or an experimental ML workload—allocates all available cluster containers until completion, forcing all subsequent business-critical tasks into a waiting state.
- Unbounded Queue Capacities: Even when multi-queue schedulers are present, setting maximum queue capacities to 100% without hard boundaries allows burst-heavy workloads to expand and occupy the entire cluster footprint, leaving no capacity for concurrent submissions.
- Disabled Container Preemption: When preemption is disabled, the scheduler cannot gracefully reclaim containers from lower-priority, long-running jobs to serve incoming high-priority or SLA-bound workloads, resulting in indefinite queuing delays.
- Absence of User and Application Limits: Lack of per-user resource capping allows a single user submitting hundreds of concurrent tasks or multi-stage DAGs to exhaust the entire resource quota assigned to their parent department or queue.

3. Impact on System Performance

- Critical Pipeline SLA Breaches: Scheduled operational pipelines are delayed for hours while waiting for unprioritized interactive queries to finish.
- Cluster Deadlock & Bottlenecks: Dependent workflow stages stall because application master containers cannot secure even minimal allocations to initiate processing tasks.
- Degraded Tenant Isolation: One poorly written user query can destabilize cluster responsiveness for all organizational stakeholders sharing the environment.

4. Corrective Actions & Architectural Remediation

- Deploy Hierarchical Capacity Scheduling: Implement a multi-tiered Capacity Scheduler that logically divides total cluster resources among organizational departments (e.g., dedicated percentages for Production ETL, Machine Learning, and Ad-Hoc Analytics) with guaranteed minimum capacities.

- Configure Hard Maximum Capacities: Set explicit maximum capacity ceilings on exploratory and non-critical queues to prevent ad-hoc analytics or ML pipelines from expanding beyond safe operational boundaries during periods of cluster contention.
- Enable Container Preemption: Activate automated resource preemption within the scheduler. When a high-priority queue requires resources to meet its guaranteed capacity, the scheduler identifies and gracefully reclaims containers from over-allocated lower-priority queues.
- Enforce User Limits and Fair-Share Policies: Set strict minimum-user-limit percentages within individual queues to guarantee that no single user can consume more than their proportionate share of the queue when other team members submit tasks.
- Establish Workload Prioritization: Configure application-level priority mapping so production ETL pipelines automatically take precedence over general development tasks during cluster scheduling cycles.

Question 5: Outdated Data Restoration in Distributed Backup Recovery

1. Problem Statement & System Context

Disaster recovery (DR) strategies in enterprise distributed storage systems must provide strict Point-in-Time Recovery (PITR) and data consistency guarantees. When a disaster recovery drill or emergency failover restores an outdated, inconsistent, or corrupted version of the dataset, it reveals severe architectural flaws in snapshot synchronization, write quiescence, or differential backup management.

2. Root Cause & Architectural Error Analysis

- Unquiesced In-Flight Writes (Dirty Backups): Initiating backup transfers or raw file copies directly against live, mutating data paths captures an inconsistent state. Write streams buffered in application memory, operating system page caches, or database Write-Ahead Logs (WAL) are bypassed during the copy, leading to missing transaction state upon restoration.
- Desynchronization Between Metadata and Block Data: In systems like HDFS, the filesystem namespace metadata (the NameNode's directory tree and block-to-file mappings) and the physical DataNode blocks are copied at slightly different times. This timing mismatch creates a desynchronized state where restored metadata references block identifiers that do not exist or are outdated on disk.
- Broken or Incomplete Incremental Backup Chains: Incremental backup frameworks depend on a strictly ordered chain of differential snapshots applied over a base full backup. If intermediate differential manifests are omitted, corrupted, or applied out of chronological order during recovery, the system silently defaults back to a stale historical state.
- Lack of Read-Only Snapshot Isolation: Backups executed using standard distributed file copy tools directly on live source directories rather than frozen, immutable snapshots are subject to file modifications mid-copy, leading to partial writes and inconsistent restore targets.

3. Impact on System Performance

- Severe Data Loss: High-value business transactions committed between the stale backup state and the point of failure are permanently lost.
- System-Wide State Corruption: Discrepancies between filesystem metadata and block storage lead to orphan blocks, broken directory structures, and database engine startup failures.
- Prolonged Recovery Time Objectives (RTO): Identifying that restored data is stale forces operations teams to abort the recovery, troubleshoot backup manifests, and manually rebuild missing transactions, extending downtime.

4. Corrective Actions & Architectural Remediation

- Enforce Immutable, Read-Only Snapshots: Never execute backup copy utilities against live, active directories. Always generate a frozen, point-in-time, read-only filesystem snapshot before initiating data replication to disaster recovery targets.
- Implement Pre-Backup Write Quiescence: Before capturing a snapshot, coordinate with running databases and processing engines to flush all in-memory buffers, MemStores, and Write-Ahead Logs to persistent disk storage, ensuring transaction consistency.
- Synchronize Metadata and Block Storage Replication: Ensure that metadata checkpoints and corresponding block data are captured and replicated atomically under the exact same transaction ID and snapshot state to prevent namespace-to-block divergence.
- Implement Cryptographic Manifest Verification: Build automated validation workflows that verify incremental backup lineage, comparing source transaction logs and cryptographic checksums against restored blocks to ensure that no intermediate diffs are missing.
- Schedule Automated Recovery Drills: Regularly execute automated, end-to-end sandbox restoration exercises to validate that the backup strategy consistently produces an up-to-date, functional, and fully verified system state.